NESTIQ PRO

Presentation *Q&A pack*

Spoken answers for a project defence. Short enough to say out loud. Strong enough if they ask a follow-up.

DOCUMENT
Oral presentation brief

PRODUCT
Smart rental management

STACK
Vanilla JS · Firebase · simulated pay

PREPARED FOR
Ntiyiso Justin · nestiq-pro.web.app

HOW TO USE THIS

Keep the first paragraph. Use the gold box only if they probe.

Markers usually ask why it exists, who it is for, how it works, and what you would change. Speak the main answer. Do not read the follow-up unless they push.

In the room

1. Lead with the problem: WhatsApp and Excel have no record of who paid, who fixed it, or what it cost.
2. Demo one loop: tenant reports a leak → contractor completes with parts → landlord pays → both sides get a receipt. Chat is the backup demo if time is short.
3. Be honest: card/EFT does not debit a real bank; PayFast is the intended live gateway; WhatsApp is pre-filled; receipts are not SARS invoices.
4. If stuck, land on the closer sentence at the end of this pack.

01
Problem and purpose
Why NestIQ exists

02
Users and roles
Landlord, tenant, caretaker, contractor

03
Architecture
Vanilla JS and Firebase

04
Features
Jobs, payouts, rent, chat

05
Security and privacy
Rules, POPIA, passwords

06
Limits and evaluation
Bugs, testing, next month

01 — PROBLEM AND PURPOSE

Why this product exists

Start here. If they only remember one thing, it should be the problem.

Q What problem does NestIQ solve?

Landlords in South Africa often run rentals on WhatsApp, Excel, and email. Rent due dates, repairs, contractor payments, and tenant chat get lost. NestIQ puts the people who actually run a building — landlord, tenant, caretaker, contractor — into one record: who paid, who fixed it, and what it cost.

Q Who is it for?

Small-to-medium South African landlords. Tenants get a portal. Caretakers and contractors are invited in — they do not share the landlord password. It is not an enterprise ERP.

Q Why not just Excel and WhatsApp?

Spreadsheets do not notify anyone, do not give tenants a login, and do not keep a clean history of payments and repairs. WhatsApp is easy to miss. A contractor paid in cash has no receipt next to the job. NestIQ stores that loop in one place.

Q If you only had two minutes, what would you demo?

Tenant reports a leak. Contractor accepts, starts, completes with parts. Landlord pays from the completed job and downloads a receipt. If time is tight, chat with an unread badge is the backup — it is visual and fast.

02 — USERS AND ROLES

Who signs in, and what they can do

Q How does a landlord account differ from a tenant account?

A landlord signs up and owns the portfolio. Everyone else is invited. A tenant never creates a landlord account. Caretakers and contractors redeem a staff invite and land on their own dashboard, not the full rent roll.

Q How does a tenant get access?

The landlord adds the tenant, then sends a portal invite — a code like NQ-XXXXXX, or a link. The tenant signs in with their email and the code. Their account is linked to that landlord and that tenant profile in Firestore.

Q What about caretakers and contractors?

The landlord invites them from Staff. A caretaker can log issues, review urgency, and assign a contractor inside the landlord's settings. A contractor only sees assigned jobs: accept or decline, start, complete with parts and photos. They cannot mark themselves paid.

Q What can each side do that the other cannot?

The landlord sees properties, all tenants, expenses, and payouts. The tenant sees only their unit. The contractor sees only their jobs. Firestore rules enforce that — not just hidden menus.

Q What if the wrong person signs in as a landlord?

The UI hides landlord screens for tenants, but that is not the real lock. Firestore security rules are. A tenant can only read their own rows under that landlord. They cannot load another landlord's tenants even from the browser console.

> IF THEY PROBE
>
> Hiding a menu is not security. Anyone can call Firebase from DevTools. Rules on the server decide what is allowed.

03 — ARCHITECTURE

How the system is built

Q Why vanilla JavaScript instead of React or Angular?

This is a school project that had to ship as a working product. Vanilla JS and HTML pages keep it simple to host on Firebase Hosting, with no build step required for the core app. The tradeoff is a larger codebase of globals. A framework would help if a team grew.

Q Why Firebase instead of your own backend?

Firebase gives authentication, a realtime database, and file storage without writing a full custom server. That fits a student project: login, live chat, and cloud sync work out of the box.

> IF THEY PROBE
>
> We still wrote the product logic: rent status, invitations, contractor stages, payouts, receipts, unread chat, and the role split. Firebase is infrastructure, not the whole system.

Q How is data scoped so Landlord A cannot see Landlord B?

Data lives under landlords/{landlordUid}/… — tenants, payments, messages, and so on. The signed-in landlord's UID is the folder. Tenants are linked through tenantAccounts and can only read their own landlord's documents.

Q How do live updates work — for example chat?

Firestore listeners. When a tenant sends a chat, the landlord's messages listener fires, the list updates, and unread badges refresh. No page reload.

Q Where is data stored?

Signed-in use is Firestore, and Storage for photos. The browser holds the live arrays in memory while you work. The real product path is the cloud, not a spreadsheet on one PC.

Q Why South Africa, Rand, and PayFast?

Amounts are in ZAR. Banks in the EFT step are FNB, Standard Bank, Absa, Nedbank, Capitec. Lease copy follows local notice habits. PayFast is the intended live checkout. For this school project the lecturer accepted a simulated in-app wizard — we do not charge a real card.

04 — FEATURES THEY WILL POKE AT

What to say about the product surface

Q How is rent marked paid or overdue?

Each tenant has a monthly due day. If a payment exists for the current month, status is paid. If today is after the due day and there is no payment, it is overdue. Otherwise it is due soon. The dashboard and the tenant home both use that status.

Q How do public listings and applications become a tenant?

A vacant property can be listed. An applicant submits a public application. The landlord reviews it, approves it, and that creates a tenant profile. Then they send a portal invite so the person can sign in.

Q How does a maintenance job actually move?

The tenant reports a fault with a suggested priority. The landlord or caretaker sets the official urgency the contractor sees. The job is assigned. The contractor accepts or declines, starts work, then completes with a description, parts, labour, and photos. The landlord board still reads Open / In progress / Done; the contractor portal owns the stages.

Q How do contractors get paid?

Only after the job is complete. The landlord or caretaker opens Pay contractor — the same card / EFT wizard as rent, with the vendor's name and masked bank details. On success we store paid amount, reference, receipt number, and who paid. The contractor sees Awaiting payment or Paid. They cannot mark themselves paid; Firestore rules block those fields on their writes.

> IF THEY PROBE
>
> This is a school simulation. The wizard does not debit FNB or Capitec. PayFast is the intended live gateway. The important product rule is still real: the contractor cannot certify their own payout.

Q What is on the receipt?

A NestIQ PDF: amount, receipt number, payment reference, contractor, job, property, who paid, and the date. Same idea after a tenant pays rent. It is proof of payment in the app, not a SARS tax invoice — say that out loud if they ask.

Q How does in-app chat work, including unread badges?

Messages sit in that landlord's messages collection, tagged with tenantId and fromRole. Unread means read is false. Opening the chat marks those messages read.

> IF THEY PROBE
>
> We had a bug where the landlord list was overwritten in memory after a cloud sync. We also briefly marked tenant messages read as soon as they arrived. Both are fixed: keep one shared list, and only mark read when the person opens the thread.

Q What do WhatsApp and EmailJS actually send?

WhatsApp is not fully automated. It opens WhatsApp with a pre-filled reminder and the landlord still taps send. Emails can go through EmailJS once templates are configured. We did not build an unofficial WhatsApp blaster.

Q What does the Gemini AI do, and what must the landlord still check?

On Pro, Gemini Flash can suggest rent, help screen a tenant from details the landlord typed, or summarise the portfolio. It is not a credit bureau and not legal advice. The landlord must check the output. It needs a Gemini API key in Settings.

Q How do PayFast, plans, and the trial work?

Starter is limited — about two properties and five tenants. New landlords get a 14-day Pro trial. Paid Pro is designed to go through PayFast in Rand. For this school project we are not charging real customers. In-app rent and contractor payouts use the simulated wizard the lecturer accepted.

05 — SECURITY AND PRIVACY

Trust, data, and what is still missing

Q How are passwords stored?

We do not store passwords. Firebase Authentication hashes and stores them. We only store profile fields — name, email, and the landlord or tenant link.

Q Can a tenant read another tenant's messages or payments?

No, if the rules are deployed. They can read messages and payments that belong to their tenant user ID or their tenant profile under their landlord. Another tenant's documents are not allowed.

Q Why do Firestore rules matter if the UI already hides those screens?

Anyone can call Firebase from the console. Security has to live on the server — the rules — not only in HTML. If rules were missing, a signed-in user could potentially read or write another landlord's data.

Q What personal data do you store? What about POPIA?

We store names, emails, phones, property addresses, payment records, bank details on vendor profiles, and optional photos. There is a public Terms and Privacy page. A full POPIA export-and-delete flow for a tenant on request is still the gap before a real launch.

Q What would you add before real landlords used this?

Firebase App Check so the API cannot be abused from random sites, backups, an audit log of who changed what, and a proper POPIA delete/export flow. Also a real support process and live billing only after those are in place.

Q What if tenant IDs are stored as "3" in one place and 3 in another?

Firestore treats a string and a number as different values. We hit that in chat and photos, so we now compare IDs as strings in the app and in some rules. That is a real production detail, not a textbook one.

06 — LIMITS, TESTING, AND "SO WHAT"

Be honest. Then show you know the next step.

Q What did you not finish, and why?

Live charging is not the focus of this school build — the wizard is simulated. There is no native iOS or Android app. Email needs EmailJS setup. WhatsApp is not fully automated. Offline use is limited. Staff exist as invited caretakers and contractors, not as a full multi-admin company login with shared permissions.

Q What breaks if the user is offline?

Listeners and writes need a network. Some UI might still show the last data in memory, but it is not a proper offline-first app. That would be a future piece of work, using Firestore's offline cache more deliberately.

Q If you had another month, what would you build first?

A tenant data-export and delete flow, Firebase App Check, live PayFast only after that, and tighter onboarding so a landlord can go from signup to first paid contractor job without getting stuck.

Q How did you test it?

Two browsers. Landlord in one, tenant or contractor in the other. Walk the job loop, then pay and download the receipt. For chat: send, check the unread badge, open the thread, confirm it clears.

Q What bug taught you the most?

Tenant chat was saved in Firestore, but the landlord screen replaced its message list with an empty local array after sync. The lesson: the UI must keep the same array the rest of the app reads, and unread state must not be marked read until the user actually opens the thread.

Q How would you measure success?

On-time rent rate, fewer missed tenant messages, time to record a payment, and whether tenants actually use the portal instead of only WhatsApp.

Q Who would pay, and why Starter vs Pro?

Landlords pay. Tenants are free. Starter is for one or two units. Pro is for more units, AI tools, and reminders. The tenant is the product's second user, not the customer.

Could two landlords share a team login?

Not as one shared company login. One Firebase user is one person. Caretakers and contractors are separate invited accounts with their own dashboards. Extra landlord admins with split permissions would still be more work.

IF THEY ASK YOU TO SUMMARISE NESTIQ IN ONE SENTENCE

Say this.

NestIQ is a web app where a landlord, tenant, caretaker, and contractor share one record of the building — who paid, who fixed it, and what it cost — with Firebase keeping the sides in sync.

Last-minute notes

- If you forget a technical detail, go back to the user: “The landlord sees the portfolio; everyone else only sees their work.”
- Never claim WhatsApp sends itself, that AI replaces a credit check, or that the wizard debits a real bank.
- If they ask “is this production-ready?” — say it is a working prototype with real auth, a real database, and real role rules, not a launched company.
- Demo the contractor pay + receipt. It is concrete, visual, and shows you did not let the contractor mark themselves paid.